BRAC University — CSE 221: Algorithms

Semester Final Exam · Spring 2026

Complete Model Solutions — Set A & Set B

SET A — Model Solutions

Question 1 (CO1) — Graph: DFS, Topological Sort, SCC [10 marks]

Graph G (Set A): Nodes {1,2,3,4,5,6,7}. From the figure the edges are:

1→2, 1→3 | 2→4 | 3→4, 3→6 | 4→5 | 5→7 | 6→4

1(a) — DFS from node 1 (neighbors in numerical order) [3 marks]

We run DFS starting at 1. At each node we explore unvisited neighbors in increasing order.

Step	Action	d[v] / f[v] update
1	Visit 1	d[1]=1
2	Explore neighbor 2 → visit 2	d[2]=2
3	Explore neighbor 4 (from 2) → visit 4	d[4]=3
4	Explore neighbor 5 (from 4) → visit 5	d[5]=4
5	Explore neighbor 7 (from 5) → visit 7	d[7]=5
6	No unvisited neighbors from 7 → finish 7	f[7]=6
7	No more neighbors from 5 → finish 5	f[5]=7
8	No more neighbors from 4 → finish 4	f[4]=8
9	No more neighbors from 2 → finish 2	f[2]=9
10	Back to 1, explore neighbor 3 → visit 3	d[3]=10
11	Explore neighbor 4 (from 3) — already visited; try 6 → visit 6	d[6]=11
12	Explore neighbor 4 (from 6) — already visited → finish 6	f[6]=12
13	No more unvisited neighbors from 3 → finish 3	f[3]=13
14	No more neighbors from 1 → finish 1	f[1]=14

Summary of d[v] and f[v]:

Node	d[v]	f[v]
1	1	14
2	2	9
3	10	13
4	3	8
5	4	7
6	11	12
7	5	6

1(b) — Valid Compilation Order (Topological Sort) [3 marks]

Topological sort = nodes in **decreasing order of finish time f[v]**:

f[v] rank	Node	f[v] value
1	1	14
2	3	13
3	6	12
4	2	9
5	4	8
6	5	7
7	7	6

Valid Compilation Order: 1 → 3 → 6 → 2 → 4 → 5 → 7

Verification: Every module appears after all modules it depends on. ✓

1(c) — Cycles in G' (after adding edge 7→3) [2 marks]

Original G has edges including: 3→4→5→7. Adding 7→3 creates a back edge from 7 to 3.

Cycle 1: 3 → 4 → 5 → 7 → 3

Cycle 2: 3 → 6 → 4 → 5 → 7 → 3 (via the path through 6)

Note: 6→4 is an edge in G, so 3→6→4→5→7→3 is also a valid cycle.

1(d) — SCC Condensation / Merged Dependency Graph [2 marks]

All nodes that participate in any cycle must be merged. Nodes 3, 4, 5, 6, 7 all appear in at least one cycle. Nodes 1 and 2 are not in any cycle.

Group	Members	Role
Merged Node M	{3, 4, 5, 6, 7}	All cycle participants merged
Node 1	{1}	Independent (no cycle)
Node 2	{2}	Independent (no cycle)

Resulting Condensed Graph:

- Node 1 → Node 2 (edge 1→2)
- Node 1 → Node M (edge 1→3, since 3 ∈ M)
- Node 2 → Node M (edge 2→4, since 4 ∈ M)
- No outgoing edges from M to nodes outside M.

Condensed graph: 1 → 2 → M and 1 → M (DAG — topological order: 1, 2, M)

Question 2 (CO1/CO2/CO3) — Hospital Robot: Shortest Path [10 marks]

Set A Graph edges (P→W):

Edge	c (cost)	p (prob)
P → A	3	0.90
P → B	5	0.80
A → C	4	0.85
A → D	6	0.55
C → D	2	0.60
C → W	5	0.75
D → B	3	0.75
D → W	4	0.85

2(a) — Optimal Route by Inspection [2 marks]

Enumerate candidate routes and evaluate:

Route	Total Cost (c)	Total Traversability (p)
P→A→C→W	3+4+5 = 12	0.90×0.85×0.75 = 0.574
P→A→D→W	3+6+4 = 13	0.90×0.55×0.85 = 0.421
P→A→C→D→W	3+4+2+4 = 13	0.90×0.85×0.60×0.85 = 0.390
P→B→? (B has no outward edge shown to W)	—	—
P→A→D→B (B dead-end)	—	—

Optimal Route: P → A → C → W

Reason: Cost = 12 (lowest), Traversability = 0.574 (highest among valid paths to W). It satisfies both objectives — lower cost AND higher traversability.

2(b) — Effective Cost Formula [2 marks]

We need w(u,v) that is **smaller when c is low and p is high**.

A natural choice divides cost by probability:

w(u, v) = c(u, v) / p(u, v)

Justification: When c is small, the numerator is small → w is small. When p is large (close to 1), the denominator is large → w is small. Both effects push w in the desired direction. This keeps all weights positive so Dijkstra's algorithm can be applied directly.

2(c) — Dijkstra's Simulation with Effective Cost [4 marks]

First, compute w(u,v) = c/p for every edge:

Edge	c	p	w = c/p (rounded 2dp)
P → A	3	0.90	3.33
P → B	5	0.80	6.25
A → C	4	0.85	4.71

A → D	6	0.55	10.91
C → D	2	0.60	3.33
C → W	5	0.75	6.67
D → B	3	0.75	4.00
D → W	4	0.85	4.71

Dijkstra's Algorithm Steps:

Step	Visited	dist[P]	dist[A]	dist[B]	dist[C]	dist[D]	dist[W]	Extract
Init	{}	0	∞	∞	∞	∞	∞	P (0)
1: extract P	{P}	0	3.33	6.25	∞	∞	∞	A (3.33)
2: extract A	{P,A}	0	3.33	6.25	8.04	14.24	∞	B (6.25)
3: extract B	{P,A,B}	0	3.33	6.25	8.04	14.24	∞	C (8.04)
4: extract C	{P,A,B,C}	0	3.33	6.25	8.04	11.37	14.71	D (11.37)
5: extract D	{P,A,B,C,D}	0	3.33	6.25	8.04	11.37	14.71→13.08	W
6: extract W	Done	—	—	—	—	—	13.08	W reached

Step-by-step details:

- Extract P (dist=0): Relax P→A: 0+3.33=3.33; P→B: 0+6.25=6.25
- Extract A (dist=3.33): Relax A→C: 3.33+4.71=8.04; A→D: 3.33+10.91=14.24
- Extract B (dist=6.25): B has no outgoing edges to unvisited nodes → no relaxation
- Extract C (dist=8.04): Relax C→D: 8.04+3.33=11.37 (better than 14.24) ✓; C→W: 8.04+6.67=14.71
- Extract D (dist=11.37): Relax D→W: 11.37+4.71=16.08? No — D→W: c=4,p=0.85 → w=4.71; 11.37+4.71=16.08 > 14.71, so W stays at 14.71? Let's recheck D→W: 4/0.85=4.706; 11.37+4.706=16.08.

  Also D→B: 11.37+4.00=15.37 > 6.25, no update.
- W is reached via C→W: dist[W] = 14.71

Accurate final computation:

w(P,A)=3/0.9=3.333 w(A,C)=4/0.85=4.706 w(C,W)=5/0.75=6.667

Path P→A→C→W: effective cost = 3.333 + 4.706 + 6.667 = **14.706**

Path P→A→C→D→W: 3.333+4.706+3.333+4.706 = **16.078**

Path P→A→D→W: 3.333+10.909+4.706 = **18.948**

Final Answer: Optimal Path = P → A → C → W with total effective cost ≈ 14.71

2(d) — Verification of the Statement [2 marks]

"Minimizing the effective cost w(u,v) does not always guarantee a path with both low total cost and high overall traversability."

Logical Explanation / Counter-example:

Consider two paths:

- Path X: single edge, c=10, p=0.99 → w = 10/0.99 ≈ 10.10
- Path Y: single edge, c=1, p=0.01 → w = 1/0.01 = 100

The formula correctly prefers Path X here. However, consider:

- Path A: c=2, p=0.50 → w = 4.0
- Path B: c=3, p=0.99 → w = 3.03

The formula chooses Path B (w=3.03), which has higher cost (3 vs 2) but the formula deems it better because p is much higher. Path B is NOT the lowest-cost path. Conversely, w penalizes low probability heavily, so a path with moderate c and p might get a higher w than a path with very low c but also low p. The single scalar w cannot simultaneously optimise two independent criteria — it is a weighted trade-off, not a guarantee of optimality in both dimensions.

Conclusion: The formula w = c/p combines both metrics into one scalar, meaning a path chosen by minimising Σw may have higher total cost than another path, or lower total traversability than another path, when the two objectives are evaluated separately. Hence the statement is TRUE.

Question 3 (CO1/CO2/CO3) — Huffman Coding [10 marks]

Set A Frequencies: A=2, B=3, C=4, D=6, E=7, F=8, G=10

3(a) — Huffman Tree Construction [5 marks]

Build the Huffman tree using a min-heap (greedy):

Step	Min-Heap / Priority Queue (node: freq)	Action
Initial	A:2, B:3, C:4, D:6, E:7, F:8, G:10	Sort ascending
1	Merge A(2)+B(3) → AB:5 | Heap: C:4, AB:5, D:6, E:7, F:8, G:10	New node AB=5
2	Merge C(4)+AB(5) → CAB:9 | Heap: D:6, E:7, F:8, G:10, CAB:9	New node CAB=9
3	Merge D(6)+E(7) → DE:13 | Heap: F:8, G:10, CAB:9, DE:13	New node DE=13
4	Merge F(8)+CAB(9) → FCAB:17 | Heap: G:10, DE:13, FCAB:17	New node FCAB=17
5	Merge G(10)+DE(13) → GDE:23 | Heap: FCAB:17, GDE:23	New node GDE=23
6	Merge FCAB(17)+GDE(23) → ROOT:40 | Done	Root = 40

Tree Structure (Left=0, Right=1):

ROOT(40) → LEFT: FCAB(17), RIGHT: GDE(23)

FCAB(17) → LEFT: F(8), RIGHT: CAB(9)

CAB(9) → LEFT: C(4), RIGHT: AB(5)

AB(5) → LEFT: A(2), RIGHT: B(3)

GDE(23) → LEFT: G(10), RIGHT: DE(13)

DE(13) → LEFT: D(6), RIGHT: E(7)

Codewords (traverse from root):

Character	Frequency	Codeword	Bits used (freq × length)
F	8	00	8 × 2 = 16
C	4	010	4 × 3 = 12
A	2	0110	2 × 4 = 8
B	3	0111	3 × 4 = 12
G	10	10	10 × 2 = 20
D	6	110	6 × 3 = 18
E	7	111	7 × 3 = 21
TOTAL	**40**	**—**	**107 bits**

3(b) — Error in Friend's Implementation [3 marks]

Friend's codewords: A=000000 (6 bits), B=000001 (6), C=00001 (5), D=0001 (4), E=001 (3), F=01 (2), G=1 (1 bit)

The Mistake:

The friend assigned SHORTER codes to MORE FREQUENT characters and LONGER codes to LESS FREQUENT characters — this is the EXACT OPPOSITE of the Huffman principle. In Huffman coding, characters with higher frequencies should receive shorter codewords (e.g., G with freq=10 should get a 2-bit code, not a 1-bit code that happens to be correct here, but A with freq=2 getting 6 bits is wrong — the

high-frequency characters need short codes). The friend seems to have sorted by frequency in reverse or built the tree incorrectly.

Consequences:

1. **Increased encoded file size:** Frequent characters (high freq) now use more bits, so the total compressed size is much larger than optimal — defeating the purpose of compression.
2. **The codewords are still prefix-free** (each code is a prefix of no other), so decoding would still work correctly — it just produces a very inefficient encoding.
3. In extreme cases the 'compressed' file could be LARGER than the original uncompressed file.

3(c) — Greedy Storage within 11-bit limit [2 marks]

We want to maximize unique characters stored, prioritizing high-frequency ones. Using our Huffman codes, store characters greedily from highest frequency:

Character	Freq	Codeword Length	Cumulative Bits	Include?
G	10	2	2	✓ (2 ≤ 11)
F	8	2	4	✓ (4 ≤ 11)
E	7	3	7	✓ (7 ≤ 11)
D	6	3	10	✓ (10 ≤ 11)
B	3	4	14	✗ (14 > 11)

Characters stored within 11 bits: G, F, E, D (using 10 bits total, 1 bit remaining)

Adding any further character (B needs 4 bits → total 14 > 11) would exceed the limit.

Question 4 (CO1) — 0/1 Knapsack Dynamic Programming [10 marks]

Set A Items:

Item	Weight	Value
1	x (unknown)	3
2	3	y (unknown)
3	1	2
4	4	5

4(a) — Find x (weight of Item 1) and y (value of Item 2) [2 marks]

Given DP table row i=1 (only Item 1 available):

```
K[1] = [0, 0, 3, 3, 3, 3]
```

K[1][w] = 3 first appears at w=2 and stays 3 for w=3,4,5. This means Item 1 (value=3) fits starting from capacity 2. Therefore **x = weight of Item 1 = 2**.

Given DP table row i=2 (Items 1 and 2):

```
K[2] = [0, 0, 3, 4, 4, 7]
```

At w=3: K[2][3]=4. With Item 2 having weight 3, placing it alone gives value y. Since K[2][3] = max(K[1][3], y) = max(3, y) = 4, we get **y = 4**.

Check w=5: K[2][5]=7 = Item1(val=3,w=2) + Item2(val=4,w=3) = 3+4=7 ✓ (weight 2+3=5 ≤ 5)

Answer: x = 2, y = 4

4(b) — Why K[2][4]=4 but K[2][5]=7? [2 marks]

At capacity 4: we can include Item 1 (weight=2, value=3) OR Item 2 (weight=3, value=4) but NOT both (2+3=5 > 4). The best single choice is Item 2 alone giving value 4.

At capacity 5: we can include BOTH Item 1 (w=2) and Item 2 (w=3) since 2+3=5 ≤ 5. Combined value = 3+4 = 7. The extra unit of capacity (5 vs 4) allows both items to fit simultaneously, which is why the value jumps from 4 to 7.

4(c) — Fill Missing DP Table Entries [3 marks]

Item 3: weight=1, value=2. Item 4: weight=4, value=5.

```
DP recurrence: K[i][w] = max(K[i-1][w], K[i-1][w-weight_i] + value_i) if w≥weight_i
```

Row i=3 (add Item 3: w=1, v=2):

♥ = K[3][3] = max(K[2][3], K[2][2]+2) = max(4, 3+2) = max(4,5) = 5

♣ = K[3][5] = max(K[2][5], K[2][4]+2) = max(7, 4+2) = max(7,6) = 7

Row i=4 (add Item 4: w=4, v=5):

★ = K[4][1] = max(K[3][1], cannot fit item4 w=4 in w=1) = K[3][1] = 2

♠ = K[4][3] = max(K[3][3], cannot fit item4 w=4 in w=3) = K[3][3] = 5

◆ = K[4][4] = max(K[3][4], K[3][0]+5) = max(7, 0+5) = max(7,5) = 7

■ = K[4][5] = max(K[3][5], K[3][1]+5) = max(7, 2+5) = max(7,7) = 7

Complete DP Table:

	w=0	w=1	w=2	w=3	w=4	w=5
i=0	**0**	**0**	**0**	**0**	**0**	**0**
i=1	**0**	**0**	**3**	**3**	**3**	**3**
i=2	**0**	**0**	**3**	**4**	**4**	**7**
i=3	**0**	**2**	**3**	**♥=5**	**7**	**♣=7**
i=4	**0**	**★=2**	**3**	**♠=5**	**◆=7**	**■=7**

(Yellow cells = newly computed values)

4(d) — Traceback to Find Optimal Items [3 marks]

Traceback procedure: start at K[n][W] = K[4][5] = 7, trace upward.

If K[i][w] ≠ K[i-1][w]: item i was included. Set w = w - weight_i, move to i-1.

If K[i][w] = K[i-1][w]: item i was NOT included. Move to i-1 with same w.

Step	i	w	K[i][w]	K[i-1][w]	Decision
1	4	5	7	K[3][5]=7	Equal → Item 4 NOT included
2	3	5	7	K[2][5]=7	Equal → Item 3 NOT included
3	2	5	7	K[1][5]=3	Different (7≠3) → Item 2 INCLUDED; w=5-3=2
4	1	2	3	K[0][2]=0	Different (3≠0) → Item 1 INCLUDED; w=2-2=0
5	—	0	—	—	w=0 → STOP

Items in optimal solution: Item 1 (w=2, v=3) + Item 2 (w=3, v=4)

Total weight = 2+3 = 5 ≤ W=5 ✓ | Total value = 3+4 = 7 = K[4][5] ✓

SET B — Model Solutions

Question 1 (CO1) — Graph: DFS, Topological Sort, SCC [10 marks]

Graph G (Set B): Nodes {A, B, C, D, E, F, G}. From the figure the edges are:

A→C, A→B | C→D, C→F | B→D | D→E, D→G | F→G | E→G

1(a) — DFS from A (alphabetical order) [3 marks]

DFS visits neighbors alphabetically. Adjacency: A:{B,C}, B:{D}, C:{D,F}, D:{E,G}, E:{G}, F:{G}.

Step	Action	d[v] / f[v]
1	Visit A	d[A]=1
2	A's neighbors: B,C → visit B first (alphabetical)	d[B]=2
3	B's neighbor: D → visit D	d[D]=3
4	D's neighbors: E,G → visit E first	d[E]=4
5	E's neighbor: G → visit G	d[G]=5
6	No unvisited neighbors from G → finish G	f[G]=6
7	No more from E → finish E	f[E]=7
8	D's next neighbor G — already visited → finish D	f[D]=8
9	No more from B → finish B	f[B]=9
10	A's next neighbor: C → visit C	d[C]=10
11	C's neighbors: D (visited), F → visit F	d[F]=11
12	F's neighbor: G (visited) → finish F	f[F]=12
13	C's neighbor D visited → finish C	f[C]=13
14	No more from A → finish A	f[A]=14

Summary:

Node	d[v]	f[v]
A	1	14
B	2	9
C	10	13
D	3	8
E	4	7
F	11	12
G	5	6

1(b) — Valid Compilation Order [3 marks]

Sort by decreasing f[v]:

f[v] rank	Node	f[v] value
1	A	14

2	C	13
3	F	12
4	B	9
5	D	8
6	E	7
7	G	6

Valid Compilation Order: A → C → F → B → D → E → G

1(c) — Cycles in G' (after adding G→C) [2 marks]

Original path: C→D→E→G. Adding G→C creates back edges.

Cycle 1: C → D → E → G → C

Cycle 2: C → D → G → C (since D→G is a direct edge)

Cycle 3: C → F → G → C

1(d) — SCC Condensation [2 marks]

Nodes in cycles: C, D, E, F, G are all mutually reachable via cycles. Nodes A and B are not in any cycle.

Group	Members	Role
Merged Node M	{C, D, E, F, G}	All cycle participants
Node A	{A}	Independent
Node B	{B}	Independent

Condensed Graph:

- A → B (edge A→B)
- A → M (edges A→C, since C ∈ M)
- B → M (edge B→D, since D ∈ M)

Condensed graph: A → B → M, A → M (Topological order: A, B, M)

Question 2 (CO1/CO2/CO3) — Hospital Robot: Shortest Path [10 marks]

Set B Graph edges:

Edge	c (cost)	p (prob)
P → A	5	0.80
P → B	3	0.75
A → C	3	0.90
A → D	2	0.65
C → D	3	0.50
C → W	6	0.85
D → B	4	0.85
D → W	5	0.80

2(a) — Optimal Route by Inspection [2 marks]

Route	Total Cost	Total Traversability
P→A→C→W	5+3+6 = 14	0.80×0.90×0.85 = 0.612
P→A→D→W	5+2+5 = 12	0.80×0.65×0.80 = 0.416
P→B (no outward edge to W)	—	—
P→A→D→B (dead-end)	—	—
P→A→C→D→W	5+3+3+5 = 16	0.80×0.90×0.50×0.80 = 0.288

Optimal Route: P → A → C → W

Reason: Among valid routes reaching W, P→A→C→W has the highest traversability (0.612) though not lowest cost. P→A→D→W has lower cost (12) but much lower traversability (0.416). Considering both metrics, P→A→C→W is the best balanced option.

2(b) — Effective Cost Formula [2 marks]

Same reasoning as Set A. We need w smaller when c is low and p is high.

w(u, v) = c(u, v) / p(u, v)

2(c) — Dijkstra's with Effective Cost [4 marks]

Edge	c	p	w = c/p
P → A	5	0.80	6.25
P → B	3	0.75	4.00
A → C	3	0.90	3.33
A → D	2	0.65	3.08
C → D	3	0.50	6.00
C → W	6	0.85	7.06
D → B	4	0.85	4.71
D → W	5	0.80	6.25

Step	Extract	dist[P]	dist[A]	dist[B]	dist[C]	dist[D]	dist[W]
Init	P(0)	0	∞	∞	∞	∞	∞
1:P	A(6.25)	0	6.25	4.00	∞	∞	∞
2:B	B(4.00)	0	6.25	4.00	∞	∞	∞
B→no new	A(6.25)	0	6.25	4.00	∞	∞	∞
3:A	C,D relax	0	6.25	4.00	9.58	9.33	∞
4:D	D(9.33)	0	6.25	4.00	9.58	9.33	15.58
D relax	B,W	0	6.25	4.00→14.04	9.58	9.33	15.58
5:C	C(9.58)	0	6.25	4.00	9.58	9.33	15.58→16.64
6:W	W(15.58)	done	—	—	—	—	15.58

Step-by-step:

- Extract P(0): relax P→A=6.25, P→B=4.00
- Extract B(4.00): B has no outgoing edges to unvisited nodes
- Extract A(6.25): relax A→C: 6.25+3.33=9.58; A→D: 6.25+3.08=9.33
- Extract D(9.33): relax D→B: 9.33+4.71=14.04 > 4.00 (no update); D→W: 9.33+6.25=15.58
- Extract C(9.58): relax C→D: 9.58+6.00=15.58 > 9.33 (no update); C→W: 9.58+7.06=16.64 > 15.58 (no update)
- Extract W(15.58): done!

Traceback: W ← D (dist[D]=9.33, D→W=6.25, 9.33+6.25=15.58) ← A (dist[A]=6.25, A→D=3.08) ← P

Final Path: P → A → D → W | Total Effective Cost = 6.25 + 3.08 + 6.25 = 15.58

2(d) — Verification [2 marks]

Same logical argument as Set A applies:

Using this graph as a concrete example: Dijkstra found P→A→D→W (effective cost 15.58) as optimal. However by inspection P→A→C→W has higher traversability (0.612 vs 0.416 for P→A→D→W) but was not chosen by the effective cost criterion. Conversely P→A→D→W has lower total cost (c=12) vs P→A→C→W (c=14). The formula w=c/p picked P→A→D→W because its effective cost is lower, but this path does NOT have the highest traversability. This demonstrates the statement: minimizing Σw(c/p) does not guarantee simultaneously minimizing total cost AND maximizing traversability — it is a trade-off scalar.

Question 3 (CO1/CO2/CO3) — Huffman Coding [10 marks]

Set B Frequencies: A=1, B=2, C=4, D=6, E=8, F=9, G=10

3(a) — Huffman Tree Construction [5 marks]

Step	Priority Queue (ascending)	Action
Initial	A:1, B:2, C:4, D:6, E:8, F:9, G:10	Sort
1	Merge A(1)+B(2)→AB:3 | C:4, AB:3, D:6, E:8, F:9, G:10	AB=3
2	Merge AB(3)+C(4)→ABC:7 | D:6, ABC:7, E:8, F:9, G:10	ABC=7
3	Merge D(6)+ABC(7)→DABC:13 | E:8, F:9, G:10, DABC:13	DABC=13
4	Merge E(8)+F(9)→EF:17 | G:10, DABC:13, EF:17	EF=17
5	Merge G(10)+DABC(13)→GDABC:23 | EF:17, GDABC:23	GDABC=23
6	Merge EF(17)+GDABC(23)→ROOT:40 | Done	Root=40

Tree Structure (Left=0, Right=1):

ROOT(40) → L:EF(17), R:GDABC(23)

EF(17) → L:E(8), R:F(9)

GDABC(23) → L:G(10), R:DABC(13)

DABC(13) → L:D(6), R:ABC(7)

ABC(7) → L:AB(3), R:C(4)

AB(3) → L:A(1), R:B(2)

Character	Frequency	Codeword	Bits (freq × len)
E	8	00	8 × 2 = 16
F	9	01	9 × 2 = 18
G	10	10	10 × 2 = 20
D	6	110	6 × 3 = 18
C	4	1110	4 × 4 = 16
A	1	111100	1 × 6 = 6
B	2	111101	2 × 6 = 12
TOTAL	**40**	**—**	**106 bits**

3(b) — Error in Friend's Implementation [3 marks]

The Mistake: The friend assigned the LONGEST codes to the MOST frequent characters and SHORTEST codes to LEAST frequent characters — the exact opposite of Huffman's principle. In the friend's scheme: G (freq=10) gets codeword '1' (1 bit) and A (freq=1) gets '000000' (6 bits). While accidentally G is given a short code here, looking at A (freq=1) with 6 bits and E (freq=8) with only 3 bits — E is frequent but has a longer code than it should ideally. The core error is that the tree was built by merging highest-frequency nodes first (max-heap) instead of lowest-frequency nodes first (min-heap). This means high-frequency characters end up deeper in the tree and receive longer codes.

Consequences:

1. **Increased compressed file size** — frequent chars use more bits → less compression.

2. The codes are still prefix-free (each code is not a prefix of another) so the encoding is technically decodable, but highly inefficient.

3. The compression ratio worsens drastically; in pathological cases the 'compressed' output could be larger than the original.

3(c) — Greedy 11-bit Storage [2 marks]

Prioritise highest-frequency characters; use Set B Huffman code lengths.

Character	Freq	Code Length	Cumulative Bits	Include?
G	10	2	2	✓
F	9	2	4	✓
E	8	2	6	✓
D	6	3	9	✓
C	4	4	13	✗ (13 > 11)

Characters stored within 11 bits: G, F, E, D (using 9 bits, 2 bits remaining)

C would need 4 more bits (total 13 > 11), so it cannot be included.

Question 4 (CO1) — 0/1 Knapsack Dynamic Programming [10 marks]

Set B Items: W=5, n=4

Item	Weight	Value
1	x (unknown)	8
2	1	y (unknown)
3	4	13
4	2	9

4(a) — Find x and y [2 marks]

```
Row i=1: K[1] = [0, 0, 0, 8, 8, 8]
```

K[1][3]=8 is the first non-zero entry. Item 1 (value=8) first fits at capacity 3. Therefore **x = weight of Item 1 = 3**.

```
Row i=2: K[2] = [0, 4, 4, 8, 12, 12]
```

K[2][1]=4. Item 2 has weight=1, so at w=1 we can include it alone giving value y. Since K[2][1]=max(K[1][1], y)=max(0, y)=4, we get **y = 4**.

Check: K[2][4]=12 = Item1(v=8,w=3)+Item2(v=4,w=1) = 12, weight=4 ≤ 4 ✓

Answer: x = 3, y = 4

4(b) — Why K[2][3]=8 but K[2][4]=12? [2 marks]

At capacity 3: we can fit EITHER Item 1 (w=3, v=8) OR Item 2 (w=1, v=4) but NOT both (3+1=4 > 3). Best single choice is Item 1 alone → value 8.

At capacity 4: we can fit BOTH Item 1 (w=3) and Item 2 (w=1) since 3+1=4 ≤ 4. Combined value = 8+4 = 12. The extra unit of capacity enables taking both items simultaneously.

4(c) — Fill Missing Entries [3 marks]

Item 3: weight=4, value=13. Item 4: weight=2, value=9.

```
Recurrence: K[i][w] = max(K[i-1][w], K[i-1][w-weight_i]+value_i)
```

Row i=3 (add Item 3: w=4, v=13):

♥ = K[3][4] = max(K[2][4], K[2][0]+13) = max(12, 0+13) = 13

♣ = K[3][5] = max(K[2][5], K[2][1]+13) = max(12, 4+13) = max(12,17) = 17

Row i=4 (add Item 4: w=2, v=9):

★ = K[4][2] = max(K[3][2], K[3][0]+9) = max(4, 0+9) = 9

♠ = K[4][3] = max(K[3][3], K[3][1]+9) = max(8, 4+9) = max(8,13) = 13

◆ = K[4][4] = max(K[3][4], K[3][2]+9) = max(13, 4+9) = max(13,13) = 13

■ = K[4][5] = max(K[3][5], K[3][3]+9) = max(17, 8+9) = max(17,17) = 17

Complete DP Table:

	w=0	w=1	w=2	w=3	w=4	w=5
i=0	**0**	**0**	**0**	**0**	**0**	**0**

i=1	**0**	**0**	**0**	**8**	**8**	**8**
i=2	**0**	**4**	**4**	**8**	**12**	**12**
i=3	**0**	**4**	**4**	**8**	**♥=13**	**♣=17**
i=4	**0**	**4**	**★=9**	**♠=13**	**◆=13**	**■=17**

(Yellow = newly computed values)

4(d) — Traceback [3 marks]

Start at K[4][5]=17, trace upward:

Step	i	w	K[i][w]	K[i-1][w]	Decision
1	4	5	17	K[3][5]=17	Equal → Item 4 NOT included
2	3	5	17	K[2][5]=12	Different → Item 3 INCLUDED; w=5-4=1
3	2	1	4	K[1][1]=0	Different → Item 2 INCLUDED; w=1-1=0
4	1	0	0	K[0][0]=0	w=0 → STOP

Items in optimal solution: Item 2 (w=1, v=4) + Item 3 (w=4, v=13)

Total weight = 1+4 = 5 ≤ W=5 ✓ | Total value = 4+13 = 17 = K[4][5] ✓